SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT231

Examination - January Semester 2024

Data Structures and Algorithms I

Friday, 19 April 2024 **04:00 pm - 06:00pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This examination contains **10** questions and comprises **8** printed pages (including cover page).
2. You must answer **ALL** questions.
3. All answers must be written in the answer book.
4. This is **Closed-book** examination.

At the end of the examination.
Please ensure that you have written your examination number on each answer book used.

Failure to do so will mean that your work cannot be identified.
If you have used more than one answer book, please tie them together with the string provided.

THE UNIVERSITY RESERVES THE RIGHT NOT TO MARK YOUR SCRIPT IF YOU FAIL TO FOLLOW THESE INSTRUCTIONS.

ICT231 Copyright © 2024 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Answer the below multiple choice questions.

Question 1a

Which is **NOT** the core data type in Python?

a. List
b. Tuple
c. Dictionary
d. String
e. Class

(2 marks)

Question 1b

Which of the following are true? Select all the true statements.

a. All elements in a list must be of the same type
b. A list is mutable
c. A list may contain any type of object except another list
d. A given object may appear in a list more than once
e. There is no conceptual limit to the size of a list

(2 marks)

Question 1c

Which of the following are true of Python dictionaries? Select all the true statements.

a. A dictionary can contain any object type except another dictionary
b. All the keys in a dictionary must be of the same type
c. Dictionaries are mutable
d. Dictionaries are accessed by key
e. Dictionaries can be nested to any depth

(2 marks)

Question 1d

Consider the following sequence of statement:

m = 123456

n = m

Following the execution of above statements, state the number of objects and references created by Python.

a. 1 object, 1 reference
b. 1 object, 2 references
c. 2 objects, 1 reference
d. 2 objects, 2 references

(2 marks)

Question 1e

What is the runtime complexity of the list's built-in .append() method?

a. $O(1)$
b. $O(log\, n)$
c. $O(n)$
d. $O(n^2)$

(2 marks)

Question 2

The number of operations executed by algorithm A and B are $\mathbf{5n^2 + 20nlog_2n}$ and $\mathbf{15n^3 - 110n^2}$. Solve for $\mathbf{n_0}$ such that A is better than B for $\mathbf{n \geq n_0}$. Show the procedures.

(5 marks)

Question 3

Analyse the time complexity of the sorting algorithm shown below in the best and worst case. Describe why.

```
def sort(alist):
    n = len(alist)
    for j in range(n-1):
        count = 0
        for i in range(0, n-1-j):
            if alist[i]>alist[i+1]:
                alist[i], alist[i+1] = alist[i+1], alist[i]
                count += 1

        if 0 == count:
            break
    return alist
```

(5 marks)

Question 4

Given a collection of medical records, we assume that each record has multiple fields, including name, age, gender, etc. Among list, tuple and string in Python, which data structure would you choose to store each record? Which data structure would you choose to store the entire collection of records? Discuss why.

(5 marks)

Question 5

Consider the following Python implementation of the initialize_nested_list function.

```
def initialize_nested_list(n):
    l = [[0]] * n
    return l

list_l = initialize_nested_list(5)
list_l[0][0] = 1
print(list_l)
```

Question 5a

Interpret the code and write its output, where initialize_nested_list aims to initialize a nested list of zeros (i.e., [[0], [0], …]).

(5 marks)

Question 5b

Identify if there is any bug in the code. If yes, correct the code.

(5 marks)

Question 6

Consider the following Python implementation of the Queue ADT using a singly linked list.

```
class SLLQueue:
    class Node:
        def __init__(self, element, next):
            self.element = element
            self.next = next

    def __init__(self):
        self.head = None
        self.tail = None

    def enqueue(self, e):
        newest = self.Node(e,  next: None)
        if self.head is None:
            self.headd = newest
            self.tail = newest
        else:
            newest.next = self.head
            self.head = newest

    def dequeue(self):
        if self.head is None:
            return "Queue is empty"
        if self.head is self.tail:
            answer = self.tail.element
            self.head = self.tail = None
            return answer
        answer = self.tail.element
        node = self.head
        while node.next is not self.tail:
            node = node.next
        node.next = None
        self.tail = node
        return answer
```

Question 6a

Analyse the time complexity of the enqueue and dequeue operations. Explain your analysis.

(5 marks)

Question 6b

Describe the problem of the code that causes computational issues using the enqueue or dequeue operations. Code is **NOT** required.

(5 marks)

Question 6c

Given that my_q is an empty instance of the Queue ADT, the following operations are performed. Interpret the operations and write down the elements in my_q.

my_q.enqueue(20)

my_q.enqueue(24)

my_q.dequeue()

my_q.size()

my_q.dequeue()

my_q.enqueue(21)

my_q.enqueue(22)

(5 marks)

Question 7

Consider the following Python implementation of the find function.

```
def find(array, target):
    if not array:
        return

    num_row = len(array)
    num_col = len(array[0])

    for i in range(num_row):
        for j in range(num_col):
            # Complete code here

    return False
```

Question 7a

Given a two-dimension array and a target, complete the code to implement an algorithm in Python to find if the target is in the array.

(10 marks)

Question 7b

Analyse the time complexity of the find algorithm.

(5 marks)

Question 7c

If the elements in the array are sorted by row and by column, for example:

$$\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$$

Describe how to improve the find algorithm so that the time complexity can be optimized to O(n).

(5 marks)

Question 8

Complete the code below to implement an algorithm in Python to remove a node in a doubly linked list called myDLL. The myDLL object has references to the head and tail of the linked list. Note that each node object has a next reference, a previous reference, and an element reference.

```
def remove_node_from_DLL(myDLL, node):
    element = node.element
    # Complete code here

    return element
```

(10 marks)

Question 9

Complete the code below to implement an algorithm in Python to remove a node in a singly linked list called mySLL. The mySLL object has references to the head and tail of the linked list. Note that each node object has a next reference and an element reference.

```
def remove_node_from_SLL(mySLL, node):
    element = node.element
    # Complete code here

    return element
```

(10 marks)

Question 10

Based on the tree shown below in Figure Q10, answer the following questions.

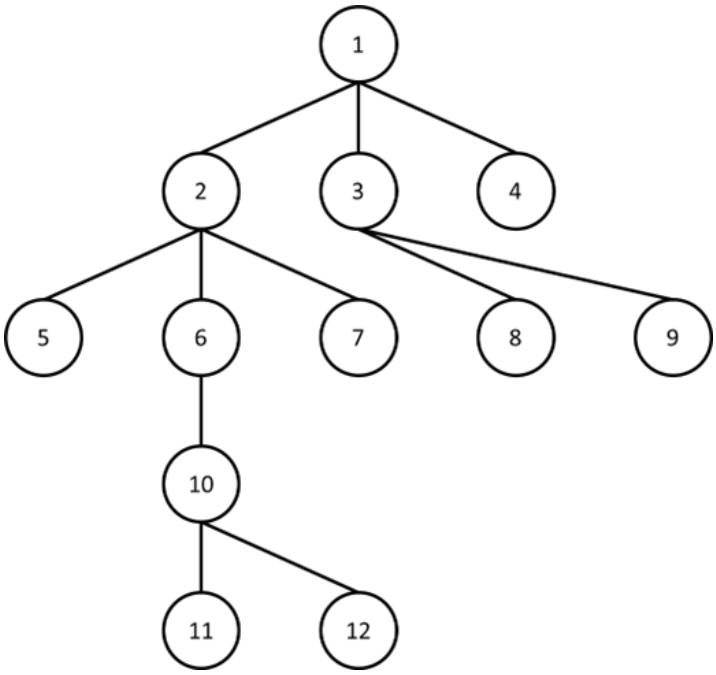

Figure Q10

Question 10a

How many internal nodes are there in the tree and what are they?

(2 marks)

Question 10b

What is the degree of node 10?

(2 marks)

Question 10c

What is the depth of node 9?

(2 marks)

Question 10d

What are the siblings of node 2?

(2 marks)

Question 10e

What is the overall order of visits in preorder traversal? Analyze its time complexity.

(2 marks)

----- END OF PAPER -----